

## Module #3 Introduction to OOPS Programming

### Practical

#### 1. Introduction to C++

Q.1] First C++ Program: Hello World

1] Write a simple C++ program to display "Hello, World!".

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    cout << "Hello, World!" << endl;  
    return 0;  
}
```

Q.2] Understand the basic structure of a C++ program, including #include, main(), and cout.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    cout << "Hello, World!";  
    return 0;  
}
```

| Part                   | Description                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| #include<br><iostream> | This is a <b>preprocessor directive</b> that tells the compiler to include the input-output stream library. It allows the use of cout and cin. |

| Part                                        | Description                                                                                                                                                                                   |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>using namespace std;</code>           | The <code>std</code> namespace contains all standard C++ library functions. This line lets you use names like <code>cout</code> and <code>cin</code> without writing <code>std::cout</code> . |
| <code>int main()</code>                     | The <b>main function</b> is where the program starts running. Every C++ program must have a <code>main()</code> function.                                                                     |
| <code>{ ... }</code>                        | Curly braces define the <b>body</b> of the main function. All executable code goes inside.                                                                                                    |
| <code>cout &lt;&lt; "Hello, World!";</code> | <code>cout</code> is used to <b>display output</b> on the screen. The operator <code>&lt;&lt;</code> sends data to the output stream.                                                         |
| <code>return 0;</code>                      | Returns control to the operating system. 0 means the program ran successfully.                                                                                                                |

## 2 Basic Input/Output

Q.1] Write a C++ program that accepts user input for their name and age and then displays a personalized greeting.

Ans.

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string name;
    int age;
    cout << "Please enter your name: ";
    getline(cin, name);
    cout << "Please enter your age: ";
    cin >> age;
    cout << "Hello, " << name << "! You are " << age << " years old." << endl;
    return 0;
}
```

Q.2] Practice input/output operations using cin and cout.

Ans.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    int number;
```

```
    double decimal;
```

```
    char letter;
```

```
    string text;
```

```
    cout << "Enter an integer number: ";
```

```
    cin >> number; // Input integer
```

```
    cout << "Enter a decimal number: ";
```

```
    cin >> decimal; // Input double
```

```
    cout << "Enter a single character: ";
```

```
    cin >> letter; // Input character
```

```
    cout << "Enter a word (no spaces): ";
```

```
    cin >> text; // Input a single word string
```

```
    cout << "\nYou entered integer: " << number << endl;
```

```
    cout << "You entered decimal: " << decimal << endl;
```

```
    cout << "You entered character: " << letter << endl;
```

```
cout << "You entered word: " << text << endl;

return 0;

}
```

### **3. POP vs. OOP Comparison Program**

Q.1] Write two small programs: one using Procedural Programming (POP) to calculate the area of a rectangle, and another using Object-Oriented Programming (OOP) with a class and object for the same task.

Ans.

```
#include <iostream>

using namespace std;

int calculateArea(int length, int width) {
    return length * width;
}

int main() {
    int length, width;
    cout << "Enter length of the rectangle: ";
    cin >> length;
    cout << "Enter width of the rectangle: ";
    cin >> width;

    int area = calculateArea(length, width);
    cout << "Area of the rectangle: " << area << endl;

    return 0;
}
```

```
#include <iostream>

using namespace std;

class Rectangle {
private:
    int length;
    int width;

public:
    Rectangle(int l, int w) {
        length = l;
        width = w;
    }
    int getArea() {
        return length * width;
    }
};

int main() {
    int length, width;
    cout << "Enter length of the rectangle: ";
    cin >> length;
    cout << "Enter width of the rectangle: ";
    cin >> width;
    Rectangle rect(length, width);
    cout << "Area of the rectangle: " << rect.getArea() << endl;
    return 0;
}
```

Q.2] Highlight the difference between POP and OOP approaches.

Ans.

| Aspect           | Procedural Programming (POP)                                                     | Object-Oriented Programming (OOP)                                                                      |
|------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Basic Concept    | Focuses on functions, procedures, and sequences of operations to manipulate data | Focuses on objects that contain data (attributes) and behavior (methods) that interact with each other |
| Data Handling    | Data is typically stored in global variables or passed explicitly to functions   | Data is encapsulated within objects, restricting direct access and promoting data hiding               |
| Modularity       | Achieved through reusable functions and procedures                               | Achieved through classes and objects, encouraging modular, reusable, and extendable code               |
| Approach         | Top-down: Breaking down tasks into functions or procedures                       | Bottom-up: Building systems by creating and combining objects                                          |
| Focus            | Mainly on how to perform tasks (procedure-oriented)                              | Mainly on what the system is (modeling real-world entities as objects)                                 |
| Code Reusability | Limited; reusing code often requires copying or redesigning functions            | High; uses inheritance and polymorphism for reusability and extension                                  |

| Aspect                       | Procedural Programming (POP)                                        | Object-Oriented Programming (OOP)                                         |
|------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------|
| Data Security                | No inherent data hiding; data can be accessed and modified globally | Supports data hiding through access modifiers (private,protected,public)  |
| Maintenance                  | Can become complex and harder to maintain as systems grow           | Easier to maintain and extend because of encapsulation and modular design |
| Size and Complexity          | Suitable for small to medium-sized programs                         | Better suited for large, complex, and scalable systems                    |
| Inheritance and Polymorphism | Not supported                                                       | Supported, enabling code reuse and flexible design                        |

#### 4. Setting Up Development Environment

Q.1] Write a program that asks for two numbers and displays their sum. Ensure this is done after setting up the IDE (like Dev C++ or CodeBlocks).

Ans. Setting up Dev C++

1. Download Dev C++ from a trusted source such as SourceForge.
2. Run the installer, accept the license, and complete the installation with default options.
3. Open Dev C++.
4. To create a new program:
  - Go to File → New → Source File.
  - Write your code in the editor window.
5. Save your program with a .cpp extension (e.g., sum.cpp).
6. Compile and run the program by pressing F11 or selecting Execute → Compile & Run.

```

#include <iostream>

using namespace std;

int main() {
    int num1, num2, sum;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    sum = num1 + num2;

    cout << "Sum of the two numbers is: " << sum << endl;

    return 0;
}

```

Q.2] Help students understand how to install, configure, and run programs in an IDE.

Ans. To help students understand how to install, configure, and run programs in an IDE, begin by choosing the right IDE for the programming language, installing necessary prerequisites, setting up the IDE, and then configuring it based on project needs. Here's a step-by-step overview with relevant details and best practices.

### **Installing an IDE**

- Download the installer for a suitable IDE (e.g., Visual Studio for C++, IntelliJ IDEA for Java, PyCharm for Python, or Visual Studio Code for versatility).



- Install any language runtime or SDK required before opening the IDE (e.g., install Java before using IntelliJ IDEA, Python before using PyCharm).
- Run the IDE installer and follow on-screen instructions to complete installation.

### **Configuring the IDE**

- Upon launch, select workloads or components for the intended programming language (e.g., Desktop development with C++ in Visual Studio).
- Configure language-specific settings and workspace folders as needed. This includes setting up your project structure, choosing build tools, and enabling version control integrations if available (like Git).
- Customize the IDE with extensions or plugins relevant to the project. Most modern IDEs allow installation of helpful tools directly from their interface.

### **Running Programs In an IDE**

- Create a new project (or open an existing one) in your IDE and write your code in the provided editor.
- Use the integrated build, run, and debug commands. Most IDEs offer buttons or menu commands labeled "Run," "Build," or "Debug" to perform these tasks.
- Errors are typically shown in a dedicated output window, and you can use the built-in debugger to step through code and analyze bugs line by line.

### **Key Features and Tips**

- IDEs feature syntax highlighting, auto-completion, integrated terminal/console, and real-time bug checks to streamline programming.
- Learning keyboard shortcuts can greatly speed up workflow within an IDE.
- Extensions and plugins can enhance productivity, allowing integration of version control, linters, and other utilities.

## 2. Variables, Data Types, and Operators

### 1. Variables and Constants

Q.1] Write a C++ program that demonstrates the use of variables and constants. Create variables of different data types and perform operations on them.

Ans.

```
#include <iostream>

using namespace std;

int main() {
    int num1 = 10;
    float num2 = 5.5;
    double num3 = 2.3456;
    char letter = 'A';
    bool isTrue = true;
    const int CONSTANT_VALUE = 100;
    int sum = num1 + CONSTANT_VALUE;
    float product = num2 * num3;

    cout << "Integer: " << num1 << endl;
    cout << "Float: " << num2 << endl;
    cout << "Double: " << num3 << endl;
    cout << "Char: " << letter << endl;
    cout << "Bool: " << isTrue << endl;
    cout << "Constant: " << CONSTANT_VALUE << endl;
    cout << "Sum: " << sum << endl;
    cout << "Product: " << product << endl;
    return 0;
}
```

Q.2] Understand the difference between variables and constants.

Ans.

```
#include <iostream>

using namespace std;

int main() {
    int variable = 10;
    variable = 20;

    const int constant = 30;

    cout << "Variable: " << variable << endl;
    cout << "Constant: " << constant << endl;
    return 0;
}
```

## 2. Type Conversion

Q.1] Write a C++ program that performs both implicit and explicit type conversions and prints the results.

Ans.

```
#include <iostream>

using namespace std;

int main() {

    int intNum = 8;
    double doubleNum = intNum;
    char ch = 'A';
    int asciiValue = ch;
```

```

double pi = 3.14159;
int intPi = (int)pi;

int x = 7;
float y = static_cast<float>(x);

cout << "Implicit int to double: " << doubleNum << endl;
cout << "Implicit char to int (ASCII of 'A'): " << asciiValue << endl;
cout << "Explicit double to int (C-style cast): " << intPi << endl;
cout << "Explicit int to float (static_cast): " << y << endl;

return 0;
}

```

Q.2] Type casting in C++.

Ans.

```

#include <iostream>

int main() {
    int num_int = 10;
    double num_double = num_int;
    std::cout << "Implicit conversion: " << num_double << std::endl;

    float val_float = 5.5f;
    int val_int = 3;
    float result = val_float + val_int;
    std::cout << "Implicit conversion in arithmetic: " << result << std::endl;
    return 0;
}

```

### 3. Operator Demonstration

Q.1] Write a C++ program that demonstrates arithmetic, relational, logical, and bitwise operators. Perform operations using each type of operator and display the results.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int a = 12, b = 5;
```

```
    bool x = true, y = false;
```

```
    cout << "Arithmetic Operators:" << endl;
```

```
    cout << "a + b = " << (a + b) << endl;
```

```
    cout << "a - b = " << (a - b) << endl;
```

```
    cout << "a * b = " << (a * b) << endl;
```

```
    cout << "a / b = " << (a / b) << endl;
```

```
    cout << "a % b = " << (a % b) << endl;
```

```
    cout << "\nRelational Operators:" << endl;
```

```
    cout << "a == b: " << (a == b) << endl;
```

```
    cout << "a != b: " << (a != b) << endl;
```

```
    cout << "a > b: " << (a > b) << endl;
```

```
    cout << "a < b: " << (a < b) << endl;
```

```
    cout << "a >= b: " << (a >= b) << endl;
```

```
    cout << "a <= b: " << (a <= b) << endl;
```

```

cout << "\nLogical Operators:" << endl;
cout << "x && y: " << (x && y) << endl;
cout << "x || y: " << (x || y) << endl;
cout << "!x: " << (!x) << endl;

cout << "\nBitwise Operators:" << endl;
cout << "a & b: " << (a & b) << endl;
cout << "a | b: " << (a | b) << endl;
cout << "a ^ b: " << (a ^ b) << endl;
cout << "~a: " << (~a) << endl;
cout << "a << 1: " << (a << 1) << endl;
cout << "a >> 1: " << (a >> 1) << endl;

return 0;
}

```

Q.2] Reinforce understanding of different types of operators in C++.

Ans. C++ provides several types of operators, each serving a distinct role in programming tasks. These types include arithmetic, relational (comparison), logical, bitwise, assignment, and conditional operators.

### Types of C++ Operators

Arithmetic Operators: Perform mathematical calculations.

- Examples: + (addition), - (subtraction), \* (multiplication), / (division), % (modulus), ++ (increment), -- (decrement).
- Usage: `int sum = a + b;`

Relational (Comparison) Operators: Compare two values and return a boolean result.

- Examples: == (equal), != (not equal), > (greater), < (less), >= (greater or equal), <= (less or equal).

- Usage: if (a > b)

Logical Operators: Operate on boolean values, controlling conditional logic.

- Examples: && (logical AND), || (logical OR), ! (logical NOT).
- Usage: if (x && y)

Bitwise Operators: Directly manipulate bits in integral data types.

- Examples: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift).
- Usage: int result = a & b;

Assignment Operators: Assign or update values in variables.

- Examples: =, +=, -=, \*=, /=, %= and bitwise assignment variations like &=, |=.
- Usage: x += 5;

Conditional (Ternary) Operator: Evaluates a condition and returns one of two values.

- Example: condition ? value1 : value2.
- Usage: int max = (a > b) ? a : b;

### 3. Control Flow Statements

#### 1. Grade Calculator

Q.1] Write a C++ program that takes a student's marks as input and calculates the grade based on if-else conditions.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int marks;
```

```
    cout << "Enter the student's marks (0-100): ";
```

```
    cin >> marks;
```

```

if (marks >= 90 && marks <= 100) {
    cout << "Grade: A+" << endl;
} else if (marks >= 80) {
    cout << "Grade: A" << endl;
} else if (marks >= 70) {
    cout << "Grade: B" << endl;
} else if (marks >= 60) {
    cout << "Grade: C" << endl;
} else if (marks >= 50) {
    cout << "Grade: D" << endl;
} else if (marks >= 0) {
    cout << "Grade: F" << endl;
} else {
    cout << "Invalid marks entered!" << endl;
}

return 0;
}

```

Q.2] conditional statements (if-else).

Ans.

```

#include <iostream>

using namespace std;

```

```

int main() {
    int number;

    cout << "Enter an integer: ";
}

```



```

cin >> number;

if (number > 0) {
    cout << "The number is positive." << endl;
} else if (number < 0) {
    cout << "The number is negative." << endl;
} else {
    cout << "The number is zero." << endl;
}

return 0;
}

```

## 2. Number Guessing Game

Q.1] Write a C++ program that asks the user to guess a number between 1 and 100. The program should provide hints if the guess is too high or too low. Use loops to allow the user multiple attempts.

Ans.

```

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0));
    int secretNumber = rand() % 100 + 1;
    int guess;
    int attempts = 0;

```

```

cout << "Guess the number (between 1 and 100):" << endl;

while (true) {
    cout << "Enter your guess: ";
    cin >> guess;
    attempts++;

    if (guess == secretNumber) {
        cout << "Congratulations! You guessed the correct number in " <<
attempts << " attempts." << endl;
        break;
    } else if (guess > secretNumber) {
        cout << "Too high! Try again." << endl;
    } else {
        cout << "Too low! Try again." << endl;
    }
}

return 0;
}

```

Q.2] Understand while loops and conditional logic.

Ans.

```
#include <iostream>
```

```

int main() {
    int count = 0;
    while (count < 5) {
        std::cout << "Count: " << count << std::endl;
    }
}

```

```

        count++;
    }
    return 0;
}

```

### 3. Multiplication Table

Q.1] Write a C++ program to display the multiplication table of a given number using a for loop.4

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {
    int num;
    cout << "Enter a number to display its multiplication table: ";
    cin >> num;

    for (int i = 1; i <= 10; ++i) {
        cout << num << " * " << i << " = " << num * i << endl;
    }

    return 0;
}

```

Q.2] using loops.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {

```

```

int n, sum = 0;

cout << "Enter positive numbers to add (enter a negative number to stop):"
<< endl;

while (true) {
    cin >> n;
    if (n < 0) {
        break;
    }
    sum += n;
}

cout << "Sum of entered positive numbers: " << sum << endl;
return 0;
}

```

#### 4. Nested Control Structures

Q.1] Write a program that prints a right-angled triangle using stars (\*) with a nested loop.

Ans.

```

#include <iostream>
using namespace std;

int main() {
    int rows;
    cout << "Enter the number of rows: ";
    cin >> rows;

    for (int i = 1; i <= rows; ++i) {

```

```

        for (int j = 1; j <= i; ++j) {
            cout << "**";
        }
        cout << endl;
    }

    return 0;
}

```

Q.2] Nested control structures.

Ans.

```

#include <iostream>
using namespace std;

int main() {
    int age = 25;
    bool isStudent = true;

    if (age >= 18) {
        cout << "You are an adult.";
        if (isStudent) {
            cout << " You are a student.";
        } else {
            cout << " You are not a student.";
        }
    } else {
        cout << "You are not an adult.";
    }
}

```

```

return 0;

}

#include <iostream>

using namespace std;

int main() {
    int rows = 3, columns = 3;
    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= columns; ++j) {
            cout << i * j << " ";
        }
        cout << endl;
    }
    return 0;
}

```

## 4. Functions and Scope

### 1. Simple Calculator Using Functions

Q.1] Write a C++ program that defines functions for basic arithmetic operations (add, subtract, multiply, divide). The main function should call these based on user input.

Ans.

```

#include <iostream>

using namespace std;

float add(float a, float b) {
    return a + b;
}

float subtract(float a, float b) {

```

```
    return a - b;
}
```

```
float multiply(float a, float b) {
    return a * b;
}
```

```
float divide(float a, float b) {
    if (b == 0) {
        cout << "Error: Division by zero is undefined." << endl;
        return 0;
    }
    return a / b;
}
```

```
int main() {
    float num1, num2;
    char op;

    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter an operator (+, -, *, /): ";
    cin >> op;
    cout << "Enter second number: ";
    cin >> num2;

    float result;
    switch(op) {
```

```
    case '+':
        result = add(num1, num2);
        break;
    case '-':
        result = subtract(num1, num2);
        break;
    case '*':
        result = multiply(num1, num2);
        break;
    case '/':
        result = divide(num1, num2);
        break;
    default:
        cout << "Invalid operator!" << endl;
        return 1;
}

cout << "Result: " << result << endl;

return 0;
}
```



## 2. Factorial Calculation Using Recursion

Q.1] Write a C++ program that calculates the factorial of a number using recursion.

Ans.

```
#include <iostream>

using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    int num;
    cout << "Enter a positive integer: ";
    cin >> num;

    if (num < 0) {
        cout << "Factorial is not defined for negative numbers." << endl;
    } else {
        int result = factorial(num);
        cout << "Factorial of " << num << " is: " << result << endl;
    }

    return 0;
}
```

Q.2] Understand recursion in functions.

Ans. Recursion in functions is a programming technique where a function calls itself in order to solve a problem by breaking it down into smaller, simpler subproblems. Essentially, a recursive function solves a problem by reducing it step-by-step until it reaches a base case, which is the simplest form of the problem with a known solution.

### **Key Concepts of Recursion:**

1. Base Case: The stopping condition where the recursion ends. Without this, the function would call itself indefinitely, causing a stack overflow.
2. Recursive Case: The part where the function calls itself with a simpler or smaller input, moving toward the base case.

### **How Recursion Works:**

- The function keeps calling itself with reduced input.
- Each call waits for the result of the next recursive call.
- Once the base case is reached, the recursion starts to unwind, returning results back through the chain of calls.

### **3. Variable Scope**

Q.1] Write a program that demonstrates the difference between local and global variables in C++. Use functions to show scope.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int globalVar = 50;
```

```
void display() {
```

```
    int localVar = 20;
```

```
    cout << "Inside display() function:" << endl;
```

```
    cout << "Local variable localVar = " << localVar << endl;
```

```
    cout << "Global variable globalVar = " << globalVar << endl;
```

```
}
```

```
void shadowingExample() {  
    int globalVar = 100;  
    cout << "Inside shadowingExample() function:" << endl;  
    cout << "Local variable globalVar = " << globalVar << endl;  
    cout << "Global variable accessed using ::globalVar = " << ::globalVar <<  
endl;  
}
```

```
int main() {  
    cout << "Inside main() function:" << endl;  
    cout << "Global variable globalVar = " << globalVar << endl;  
    // cout << "Local variable localVar = " << localVar << endl;  
  
    display();  
    shadowingExample();  
  
    return 0;  
}
```

Q.2] Reinforce the concept of variable scope.

Ans. Variable scope in C++ refers to the region in a program where a variable is accessible and can be used. It defines the life and visibility of that variable.

### Key Concepts:

- **Local Scope:** Variables declared inside a function or a block (curly braces {}) are local variables. These are accessible only within that function or block where they are declared. They are created when the block is entered and destroyed when it is exited.

- Global Scope: Variables declared outside all functions have global scope, meaning they can be accessed by any function in the same source file or file where they are declared.
- Shadowing: If a local variable has the same name as a global variable, the local variable "shadows" the global one within its scope. The global variable can still be accessed using the scope resolution operator

Example Recap:

- Local variables exist only within their function/block.
- Global variables exist throughout the program after declaration.
- Attempting to access a local variable outside its scope leads to compilation errors.

## 5. Arrays and Strings

### 1. Array Sum and Average

Q.1] Write a C++ program that accepts an array of integers, calculates the sum and average, and displays the results.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of elements: ";
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    cout << "Enter " << n << " integers:" << endl;
```

```
    for (int i = 0; i < n; ++i) {
```

```
        cin >> arr[i];
```

```
    }
```

```

int sum = 0;
for (int i = 0; i < n; ++i) {
    sum += arr[i];
}

double average = static_cast<double>(sum) / n;

cout << "Sum = " << sum << endl;
cout << "Average = " << average << endl;

return 0;
}

```

## 2. Matrix Addition

Q.1] Write a C++ program to perform matrix addition on two 2x2 matrices.

Ans.

```

#include <iostream>
using namespace std;

int main() {
    int matrix1[2][2], matrix2[2][2], sum[2][2];

    cout << "Enter elements of first 2x2 matrix:" << endl;
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            cout << "Enter element [" << i << "][" << j << "]: ";
            cin >> matrix1[i][j];

```

```

    }
}

cout << "Enter elements of second 2x2 matrix:" << endl;
for (int i = 0; i < 2; ++i) {
    for (int j = 0; j < 2; ++j) {
        cout << "Enter element [" << i << "][" << j << "]: ";
        cin >> matrix2[i][j];
    }
}

for (int i = 0; i < 2; ++i) {
    for (int j = 0; j < 2; ++j) {
        sum[i][j] = matrix1[i][j] + matrix2[i][j];
    }
}

cout << "Sum of the two matrices:" << endl;
for (int i = 0; i < 2; ++i) {
    for (int j = 0; j < 2; ++j) {
        cout << sum[i][j] << " ";
    }
    cout << endl;
}

return 0;
}

```

Q.2] Multi-dimensional arrays.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int matrix[3][4] = {
```

```
        {1, 2, 3, 4},
```

```
        {5, 6, 7, 8},
```

```
        {9, 10, 11, 12}
```

```
    };
```

```
    cout << "Elements of the 3x4 matrix:" << endl;
```

```
    for (int i = 0; i < 3; ++i) {
```

```
        for (int j = 0; j < 4; ++j) {
```

```
            cout << matrix[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

### 3. String Palindrome Check

Q.1] Write a C++ program to check if a given string is a palindrome (reads the same forwards and backwards).

Ans.

```
#include <iostream>
```

```
#include <string>

using namespace std;

bool isPalindrome(const string& str) {
    int left = 0;
    int right = str.size() - 1;

    while (left < right) {
        if (str[left] != str[right]) {
            return false;
        }
        left++;
        right--;
    }

    return true;
}

int main() {
    string input;
    cout << "Enter a string: ";
    getline(cin, input);

    if (isPalindrome(input)) {
        cout << "\"" << input << "\" is a palindrome." << endl;
    } else {
        cout << "\"" << input << "\" is NOT a palindrome." << endl;
    }
}
```



```

    }

    return 0;
}

Q.2] String operations.
Ans.
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Hello";
    string str2 = "World";
    string str3 = str1 + " " + str2;
    cout << "Concatenation: " << str3 << endl;

    cout << "Length of str3: " << str3.length() << endl;

    cout << "First character of str3: " << str3[0] << endl;

    string sub = str3.substr(6, 5);
    cout << "Substring: " << sub << endl;

    size_t pos = str3.find("World");
    if (pos != string::npos)
        cout << "\"World\" found at position: " << pos << endl;
    else

```

```

        cout << "\"World\" not found." << endl;

str3.replace(pos, 5, "C++");
cout << "After replace: " << str3 << endl;

if (str1 == "Hello")
    cout << "str1 is equal to Hello" << endl;

return 0;
}

```

## 6. Introduction to Object-Oriented Programming

### 1. Class for a Simple Calculator

Q.1] Write a C++ program that defines a class Calculator with functions for addition, subtraction, multiplication, and division. Create objects to use these functions.

Ans.

```

#include <iostream>

using namespace std;

```

```

class Calculator {
public:
    double add(double a, double b) {
        return a + b;
    }

    double subtract(double a, double b) {
        return a - b;
    }
}

```

```

double multiply(double a, double b) {
    return a * b;
}

double divide(double a, double b) {
    if (b != 0)
        return a / b;
    else {
        cout << "Error! Division by zero." << endl;
        return 0;
    }
}

};

int main() {
    Calculator calc;
    double num1, num2;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    cout << "Addition: " << calc.add(num1, num2) << endl;
    cout << "Subtraction: " << calc.subtract(num1, num2) << endl;
    cout << "Multiplication: " << calc.multiply(num1, num2) << endl;
    cout << "Division: " << calc.divide(num1, num2) << endl;

    return 0;
}

```

Q.2] Introduce basic class structure.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
class Car {
```

```
public:
```

```
    string brand;
```

```
    string model;
```

```
    int year;
```

```
    void displayInfo() {
```

```
        cout << "Brand: " << brand << ", Model: " << model << ", Year: " << year  
<< endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Car myCar;
```

```
    myCar.brand = "Toyota";
```

```
    myCar.model = "Corolla";
```

```
    myCar.year = 2020;
```

```
    myCar.displayInfo();
```

```
    return 0;
```

```
}
```

## 2. Class for Bank Account

Q.1] Create a class Bank Account with data members like balance and member functions like deposit and withdraw. Implement encapsulation by keeping the data members private.

Ans.

```
#include <iostream>
```

```
using namespace std;
```

```
class BankAccount {
```

```
private:
```

```
    double balance;
```

```
public:
```

```
    BankAccount(double initialBalance) {
```

```
        if (initialBalance < 0) {
```

```
            balance = 0;
```

```
            cout << "Initial balance invalid. Set to 0." << endl;
```

```
        } else {
```

```
            balance = initialBalance;
```

```
        }
```

```
    }
```

```
    void deposit(double amount) {
```

```
        if (amount > 0) {
```

```
            balance += amount;
```

```
            cout << "Deposit successful. Current balance: " << balance << endl;
```

```
        } else {
```

```
            cout << "Invalid deposit amount." << endl;
```

```
        }
```

```
}
```

```
void withdraw(double amount) {  
    if (amount <= balance && amount > 0) {  
        balance -= amount;  
        cout << "Withdrawal successful. Current balance: " << balance << endl;  
    } else {  
        cout << "Withdrawal failed: Insufficient balance or invalid amount." <<  
endl;  
    }  
}
```

```
double getBalance() {  
    return balance;  
}  
};
```

```
int main() {  
    BankAccount account(1000);  
  
    account.deposit(500);  
    account.withdraw(200);  
    account.withdraw(2000);  
  
    cout << "Final balance: " << account.getBalance() << endl;  
  
    return 0;  
}
```

Q.2] Understand encapsulation in classes.

Ans. Encapsulation in classes is the object-oriented programming principle of bundling data (attributes) and the methods (functions) that operate on that data into a single unit, usually a class. It restricts direct access to some of an object's components, typically by making data members private, and provides public methods to access or modify that data safely.

**Key Aspects of Encapsulation:**

- **Data Hiding:** Internal data members are kept private and cannot be accessed directly from outside the class.
- **Controlled Access:** Public methods (getters and setters) provide controlled, validated access to private data.
- **Improved Security:** Prevents accidental or unauthorized modification of object data.
- **Modularity and Maintenance:** Encapsulation separates the interface (public methods) from implementation (private data), allowing internal changes without affecting external code.

Example Concept:

A class like Bank Account hides the balance variable by declaring it private and only allows deposits and withdrawals through public member functions, ensuring only valid operations affect the balance.

```
#include <iostream>
```

```
using namespace std;
```

```
class Employee {
```

```
private:
```

```
    int salary;
```

```
public:
```

```
    void setSalary(int s) {
```

```
        if (s > 0)
```

```
            salary = s;
```

```

        else
            cout << "Invalid salary!" << endl;
    }

    int getSalary() {
        return salary;
    }
};

int main() {
    Employee e1;
    e1.setSalary(50000);
    cout << "Employee Salary: " << e1.getSalary();
    return 0;
}

```

### 3. Inheritance Example

Q.1] Write a program that implements inheritance using a base class Person and derived classes Student and Teacher. Demonstrate reusability through inheritance.

Ans.

```

#include <iostream>
#include <string>
using namespace std;

```

```

class Person {
protected:
    string name;

```



```
    int age;
public:
    Person(string n, int a) : name(n), age(a) {}
    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};
```

```
class Student : public Person {
    int studentID;
public:
    Student(string n, int a, int id) : Person(n, a), studentID(id) {}
    void display() {
        Person::display();
        cout << "Student ID: " << studentID << endl;
    }
};
```

```
class Teacher : public Person {
    string subject;
public:
    Teacher(string n, int a, string sub) : Person(n, a), subject(sub) {}
    void display() {
        Person::display();
        cout << "Subject: " << subject << endl;
    }
};
```

```

int main() {
    Student s("Alice", 20, 101);
    Teacher t("Bob", 45, "Math");

    s.display();
    cout << endl;
    t.display();

    return 0;
}

```

Q.2] The concept of inheritance.

Ans. Inheritance is a fundamental concept in object-oriented programming (OOP) where a new class (called the derived or child class) is created based on an existing class (called the base or parent class). The derived class inherits the attributes and methods of the base class, allowing code reuse and establishing an "is-a" relationship between the classes. This promotes code extensibility and modular design, enabling developers to build complex hierarchies and reuse existing code efficiently.

- The base class contains common properties and methods.
- The derived class inherits those properties and methods and can add new features or override existing ones.

```

#include <iostream>

using namespace std;

class Person {
public:
    string name;
    int age;

```

```
Person(string n, int a) : name(n), age(a) {}  
void display() {  
    cout << "Name: " << name << ", Age: " << age << endl;  
}  
};  
  
class Student : public Person {  
    int studentID;  
public:  
    Student(string n, int a, int id) : Person(n, a), studentID(id) {}  
    void display() {  
        Person::display();  
        cout << "Student ID: " << studentID << endl;  
    }  
};  
  
int main() {  
    Student s("Alice", 20, 101);  
    s.display();  
    return 0;  
}
```